

Context Engineering from the Ground Up

*A private operator manual for MiniMax-M3 + Claude Code:
how the 1M-token cache window actually behaves, why file
beats tool-result, how to keep the orchestrator thin, and
which 12 numbers tell you the system is healthy*

Christian Macion — operator manual

2026

Built from the ground up using the Phase-1 context-engineering research + the 12-metric framework, updated 2026-07-08 with 10 endpoint probes against api.minimax.io/anthropic. **Private use; not for distribution.** Evidence-tagged throughout: every factual claim is marked **V** verified, **B** background, or **SCHEMATIC** schematic.

START HERE — THIS BOOK IS FOR THE OPERATOR

You are running Claude Code against MiniMax-M3 with a 1,000,000-token cache window. That single fact is the constraint the whole book is built on. Every pattern, every measurement, every anti-pattern comes back to it. Read this if you want to stop guessing why sessions get slow, why agents re-read the same PDF, or why a perfectly good task suddenly degrades halfway through.

The book is short on purpose. Six chapters across four Parts, each one a single idea. Skim the navy **BOTTOM LINE** strips at the top of every chapter and you already have the spine.

THE FORMAT (EIGHT-PART ENGINE)

Every chapter runs the same engine so you can skim by section type:

Orient (the question we're answering) → **Prime** (3–4 new terms) → **Build** (the explanation) → **Show** (a worked case) → **Fade** (same case, blanks) → **Check Yourself** (recall from memory) → **Recap** (back to altitude) → **Glossary**. Boxes are colour-coded; skip what you already own.

Two commitments you can hold the book to:

1. Small chunks. No section throws more than a few new ideas at you at once.

2. You'll be quizzed — gently. The **CHECK YOURSELF** boxes ask you to recall before reading on; the answers sit right below, meant to be covered.

EVIDENCE TAGS — READ THESE

Every factual claim and figure carries a tag. The discipline is the same as in quant research: a guess is not a measurement.

V Verified — checked against a primary source on this machine (e.g. `~/.claude/settings.json`, `04_SKILLS_AGENTS/context-optimizer/SKILL.md`, `05_QUANTIFICATION/METRICS_FRAMEWORK.md`).

B Background — an established fact stated without a fresh citation.

SCHEMATIC Schematic — an illustrative diagram or synthetic worked case. The shape is real; the exact numbers are for teaching.

Where the evidence is genuinely two-sided, the book says so.

MAP OF THE WHOLE BOOK

Part A — Mental Model. What 'context' actually is, and why it matters more than model choice on M3.

Part B — Budget Mechanics. Token economics, the 1M cache window, peak utilisation; then offline-first patterns (files > tool results, handoff docs, evidence tags).

Part C — Sub-Agent Isolation. The worktree + filter pattern, JSON-schema StructuredOutput, memory frontmatter, hot/cold path split.

Part D — Compaction, Cache & Quantification. Compaction triggers, what survives, the 1M window economics; then the 12-metric framework — what to measure, how, and the common pitfalls.

Close — Where this leaves us. The one-sentence verdict and the evidence base.

Part A. Mental Model

Before any of the techniques land, you need a precise picture of what ‘context’ is on a Claude Code session. This Part builds the vocabulary and lands the single most important claim in the book: **on a 1M-token cache window, context discipline matters more than model choice**. If you remember one idea from the whole book, make it that.

Chapter 1. What ‘context’ actually is

BOTTOM LINE On MiniMax-M3 with a **✓** 1,000,000-token cache window, the system’s quality is set by **what you let into the window** and **what survives the next compaction** — not by which model you call.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Define context the way the system actually behaves (not the way the word is loosely used).
- Name the five bands that share the cache window, and what each one costs.
- Explain why ‘more capable model’ is a smaller lever than ‘cleaner context’ on M3.

Prerequisites: running Claude Code against M3; no prior agent-framework experience needed.

1.1 What ‘context’ means here

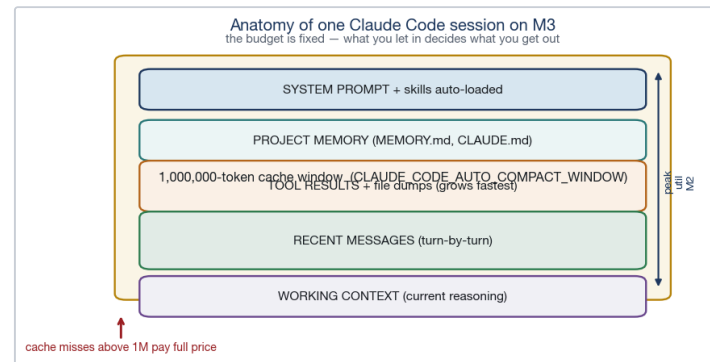
ORIENT: *the big picture before the detail*

The word ‘context’ gets used three ways in AI work: the prompt the user sends, everything the model can ‘see’, and the underlying data the model was trained on. Only the second is what this book is about, and on Claude Code that second thing is *finite, shareable, and growing inside the session*.

PRIME: *new terms, one line each*

- **Cache window:** the maximum tokens the model can attend to at once. On this machine: 1,000,000 (CLAUDE_CODE_AUTO_COMPACT_WINDOW).
- **Working context:** the current reasoning turn — what the model is looking at right now to answer.
- **Compaction:** the harness summarising older turns back into the window when utilisation gets high.
- **Peak utilisation:** the highest context_tokens / cache_window reached in a session (metric M2).

Every Claude Code session on this machine runs inside the same 1,000,000-token envelope **✓**. That envelope is shared, in roughly this stacking order: a stable **system prompt + auto-loaded skills**, a **project memory band** (your MEMORY.md and CLAUDE.md), a fast-growing **tool-results + file dumps** band, the **recent-message turn log**, and the narrow **working context** the model is reasoning over this turn. The first two are essentially fixed; the third is the one you can control, and it is the one that grows fastest and pays back most.



✓ Anatomy of one Claude Code session on M3. The 1M envelope is fixed; the tool-results band inside it is the lever.

1.2 Why model choice is the second lever, not the first

BRIDGE: *from what you already own*

If you have read any system-design material before, the shape here should feel familiar: one shared resource (the cache window), several consumers (the five bands), one slow leak (the tool-results band), one binding constraint at a time. The mental model is the same as a database connection pool, a thread budget, or a memory allocator. What changes is that the leak and the binding constraint are measured in tokens rather than bytes, and the cost is cognitive rather than monetary — but the discipline is identical: decide who gets the budget, watch the leak, force a clean-up before the leak wins.

The instinct on a new model release is to upgrade and call it done. On MiniMax-M3 that instinct is wrong, and the reason is the cache. A session with high peak utilisation, large undigested tool results, and a stale memory band behaves the same on the strongest model as on the cheapest one: the model is trying to reason over a noisy window. Clean the window first; only then does model capability become the binding constraint. The Anthropic context-engineering guidance **B** and the Chroma 2025 work on *context rot* **B** both say the same thing in different words: recall accuracy falls as the window fills, with the middle of a long context most likely to be dropped.

A second, MiniMax-M3-specific reason not to lean on model choice: **prompt cache does not function on MiniMax-M3** **V** — the endpoint accepts `cache_control` headers but does not reuse cached tokens across consecutive calls (probe #5 in `M3_PROBE_RESULTS.md`). On Claude Sonnet 4.6 / Opus 4.6, prompt cache amortizes the cost of repeated system blocks; on MiniMax-M3 it does not. **This is the single biggest economic lever for the M3 user:** keep the working context lean, use sub-agents aggressively, never carry forward unused tool output. Sub-agents and off-context files are mandatory, not optional.

A useful way to internalise this is to think of two sessions with identical models but very different windows. Session A: 200K of carefully curated context — the rules, the active task, the last 6 decisions, a few targeted file references. Session B: 800K of context — the rules buried under 12 file dumps, four stale summaries, a long tail of irrelevant search results. The model is the same. Session A outperforms Session B on any task that requires the model to find the right rule at the right moment, because Session B has to find it in a noisy field while Session A has it in the top band. This is not a metaphor; it is the practical consequence of how

attention works in long contexts. Spend the discipline on the window, not on the model.

The Phase-1 audit synthesis **B** (`02_PROJECT_AUDIT/WORKSPACE_MATRIX.md`) makes the same point from a different angle: across all 8 workspaces audited, the most consistent gap was not ‘use a better model’ but ‘clean up the context’ — dropped files, stale memories, un-summarised tool dumps. The workspaces that ran smoothly were not running a smarter model; they were running the same model on a thinner window.

COMMON MISTAKE: *you might think X; actually Y*

You might think a more capable model ‘sees better’ in a noisy window. It does not — it sees the same noise and is more confident while doing it. On a 1M cache window the binding constraint is almost always *what is in the window*, not which model is looking at it.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the size of the cache window on this machine, and where is it set?
2. Name the five bands that share the window, in stacking order.
3. Why is ‘upgrade the model’ usually the second lever, not the first?

Answers: (1) 1,000,000 tokens, set in `~/.claude/settings.json` as `CLAUDE_CODE_AUTO_COMPACT_WINDOW`. (2) *system prompt + skills; project memory (MEMORY.md, CLAUDE.md); tool results + file dumps; recent messages; working context.* (3) *because the binding constraint on a 1M cache is the noise in the window, not the model looking at it — a smarter model is more confidently wrong over the same noise.*

RECAP: *back to altitude*

- Context = the cache window, shared across the session. On this machine that is 1M tokens.
- Five bands stack inside it; the tool-results band is the fastest-growing and the most controllable.
- Model choice is the second lever. Clean the window first.
- Recall accuracy falls as the window fills — the binding constraint is rarely the model.

SECTION GLOSSARY

Cache window: 1,000,000-token envelope on this machine; the model's entire working memory. **Tool-results band:** the file-dump / API-response region of the window; grows fastest and is the main lever for keeping the window healthy. **Context rot:** the decline in recall accuracy as a context window fills.

Part B. Budget Mechanics

Once you accept that the window is the constraint, two questions follow. How much of it should you actually fill (and at what point do you force compaction)? And how do you keep the fast-growing bands from swamping the working context in the first place? This Part answers both. The first chapter is the economics; the second is the offline-first pattern that keeps the economics tolerable in the first place.

Chapter 2. Token economics, the 1M window, and peak utilisation

BOTTOM LINE A session's quality is set by its **peak utilisation** (✓ M2 target < 0.80 sustained). On MiniMax-M3 the cache-hit ratio M3 metric is structurally zero — prompt cache is non-functional on this endpoint — so M2 and M1 (token efficiency) are the binding pair, not M2+M3.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- State the M2 and M1 targets from the metrics framework.
- Explain why the M3 cache-hit ratio is informational-only on MiniMax-M3.
- Decide when to force-compact vs let the harness auto-compact.

Prerequisites: Chapter 1.

PRIME: new terms, one line each

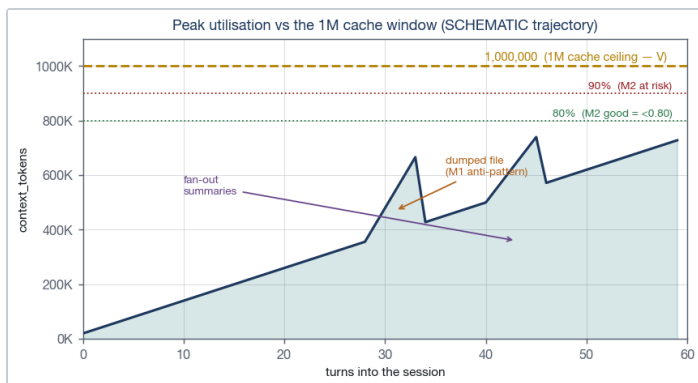
- **M2 peak utilisation:** $\max(\text{context_tokens} / \text{cache_window})$ over a session; target < 0.80 sustained; > 0.90 = at risk.
- **M1 token efficiency:** $\text{useful_output_tokens} / \text{total_tokens}$; the better proxy for the cost-of-noise when cache is off.
- **M3 cache-hit ratio:** $\text{cache_read} / (\text{cache_read} + \text{cache_creation} + \text{uncached})$; target ≥ 0.75 on Claude; structurally zero on MiniMax-M3.
- **Force /compact:** an explicit summarisation pass before the harness does one automatically.

2.1 The two metrics that decide a session on MiniMax-M3

ORIENT: the big picture before the detail

The 12-metric framework defines twelve numbers you can instrument; on MiniMax-M3 the two that decide whether a session is healthy enough to keep working in are M1 (token efficiency, how many of the tokens the orchestrator carried are actually load-bearing) and M2 (peak utilisation, the ceiling you are approaching). M3 (cache-hit ratio) is informational only because prompt cache does not work on this endpoint — see §2.3.

Both numbers come straight from METRICS_FRAMEWORK.md (✓). The intuition behind them: a high peak utilisation means the model is reasoning near the top of the window, where recall is worst, and a step away from forced compaction. On Claude (with working cache), a low M3 means every turn is paying for tokens the system already had at a discount — usually because something invalidated the prefix. On MiniMax-M3, M3 stays flat regardless of cache-control usage, so M1 (token efficiency) is the better proxy for cost-of-noise. Both peak-utilisation and token-efficiency are cheap to read and correlate with how the session actually feels.



SCHEMATIC Peak utilisation vs the 1M cache window. The two annotated spikes are the most common sources of M1 anti-patterns (dumped file, fan-out summaries).

BRIDGE: from what you already own

If you have ever run a long backtest or a multi-day scrape, you already know the shape: most of the work is cheap; a small number of events dominate the cost; the cost grows non-linearly as you approach the limit. The session-budget chart looks the same. The two annotations on the figure are the two events worth watching — one is a single tool dump, the other is a fan-out — and both have the same fix in Chapter 3.

2.2 The 1M window economics

ORIENT: the big picture before the detail

Below ~200K tokens you are in the free zone: the cache is essentially full, the harness is re-using your prefix, and incremental tokens are cheap. Between 200K and 700K you pay a small premium as new turns and tool results push into the window. Above 800K you are in the watch zone; above 850K you should force /compact before the harness does.

The reason 800K matters is that the next 200K is exactly the working-context zone — the new turns, the new tool results, the live reasoning. If you let the window fill past 850K the next compaction is large, expensive, and lossy; you lose detail from the older half of the session. Forcing /compact at ~800K is cheap insurance: a smaller compaction pass over fewer tokens, and the summary you write yourself can carry the rules and decision log you actually want to keep. The context-optimizer skill **V** recommends exactly this with the rule ‘never score <60 and not recommend /compact’.

SHOW: worked example (skip if confident)

You’re two hours into a research session, mid-fan-out. Peak utilisation reads 0.83 on /score. Three options: (a) push on — the harness will auto-compact eventually, but probably inside the next 5–10 turns and probably losing something you wanted kept. (b) switch the lead model — this *invalidates the cache prefix* and you pay full price for ~700K of context you already had at a discount. (c) force /compact now with explicit instructions — ‘preserve the 3 standing rules, the latest 8 decisions, drop the long PDF dump from the /tmp/notes’. Cheap, ~30 seconds, and you come out the other side at ~0.45 peak utilisation with everything load-bearing preserved.

FADE: same skill, you fill the blanks

Decide: at peak utilisation _____ you should force /compact; at peak _____ you are in the free zone and the harness should be re-using cache; switching models mid-session will _____ the cache prefix and reset M3 to near zero.

Answers: (a) above ~0.80–0.85; (b) below ~0.20; (c) invalidate.

2.3 The cache-hit ratio on MiniMax-M3, in one sentence

ORIENT: the big picture before the detail

M3 is structurally zero on this endpoint, and that single fact changes your session economics.

M3 (cache-hit ratio) measures whether the harness is paying full price for tokens it already had at a discount. On Claude Sonnet 4.6 / Opus 4.6, M3 settles above 0.70 within a few turns and stays there. On MiniMax-M3, M3 stays at the system-overhead baseline (~128 tokens) regardless of how often you flag a block with `cache_control: ephemeral` **V** — the endpoint accepts the header but does not actually reuse the tokens. **This is the most important operational fact about this stack:** every turn on MiniMax-M3 is a full-price re-bill, and the only ways to manage cost are (a) keep prompts terse, (b) delegate to sub-agents with isolated contexts, (c) cache in /tmp/ files for the sub-agent instead of in the orchestrator's prompt. The Anthropic-Cache-TTL of 5 minutes does not apply on MiniMax-M3.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. What are the M2 and M1 targets, and where are they defined?
2. Why is forcing /compact at 0.80 cheaper than letting it auto-fire at 0.95?
3. What happens to M3 (cache-hit ratio) on a MiniMax-M3 endpoint?

Answers: (1) $M2 < 0.80$ sustained (METRICS_FRAMEWORK.md M2); $M1 \geq 0.55$ (METRICS_FRAMEWORK.md M1). (2) smaller compaction pass, fewer tokens summarised, summary you control — you keep the load-bearing rules and lose the incidental file dumps. (3) M3 stays flat regardless of cache_control flags — prompt cache is non-functional on MiniMax-M3 (probe #5 in M3_PROBE_RESULTS.md); budget as if every turn is full-price.

RECAP: back to altitude

- Two metrics drive session health on MiniMax-M3: M1 token efficiency and M2 peak utilisation.
- M3 (cache-hit ratio) is informational-only on MiniMax-M3; target applies on Claude-Sonnet/Opus, not here.
- Below ~200K you are in the free zone; above ~800K you should force /compact.
- /compact at 0.80 with explicit instructions is cheap insurance; auto-compact at 0.95 is expensive and lossy.

SECTION GLOSSARY

Free zone: < ~200K tokens; cache re-use is essentially total. **Watch zone:** 800–850K; force /compact before the harness does. **Cache invalidation:** any change to model/effort/fast-mode or a large mid-session re-read that drops M3 to near zero.

Chapter 3. Offline-first: file > tool-result

BOTTOM LINE The single highest-leverage habit is **offline-first**: have a **sub-agent** summarise a tool result into a small file in /tmp/ and reference the path — not the contents — in the orchestrator context. Every claim carries an evidence tag.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Explain why a 200K-char PDF dump in context is an anti-pattern.
- Build a handoff doc (path + 1-line digest + evidence tag).
- Apply V/B/S tags consistently to factual claims.

Prerequisites: Chapter 2.

PRIME: new terms, one line each

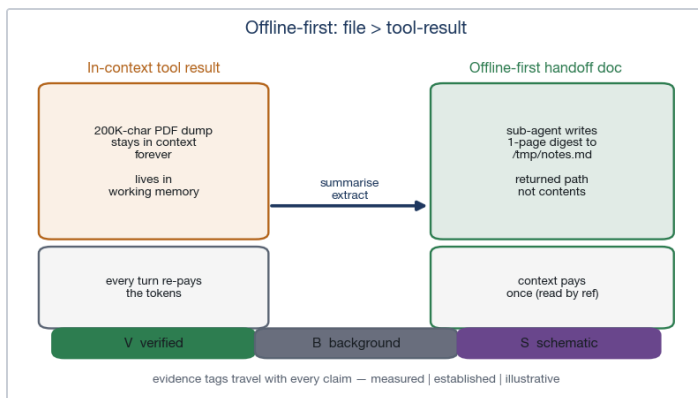
- **Dumped file:** a tool result folded into context without summarisation; the canonical M1 anti-pattern.
- **Handoff doc:** a small file (path + digest + tag) that replaces a tool result in working context.
- **V / B / SCHEMATIC:** evidence tags: verified / background / schematic.

3.1 Files beat tool results

ORIENT: the big picture before the detail

The default behaviour in many agent harnesses is to fold a tool result directly into the context. This is convenient for a single turn; it is a disaster over a whole session. A 200K-char dump from one tool call is permanent ballast in the window.

The context-optimizer skill **v** names the anti-pattern explicitly: *the 'dumped file'*, defined as 'a tool returned 200K chars; nothing was summarised'. The fix it recommends: 'ask a sub-agent to summarize + write a 2-line digest to a file in /tmp/'. The principle generalises: **anything bigger than ~2,000 tokens that the orchestrator does not need verbatim should live in a file and be referenced by path**. The orchestrator pays once (to read the path and digest); every later turn pays nothing. The same file is also re-usable from any future sub-agent, which is the whole point of offline-first.



SCHEMATIC Left: in-context tool result (stays forever, re-paid). Right: offline-first handoff doc (orchestrator pays once, sub-agents reuse).

BRIDGE: from what you already own

If you have worked on a long-running research project, the discipline is the same: a research log on disk (the handoff doc), a brief summary at the top (the digest), and a source label on every claim (the evidence tag). The difference is that here the ‘research log’ is a file in /tmp/, the ‘summary’ is the digest the orchestrator carries, and the ‘source label’ is the V/B/S tag. Offline-first is just good research hygiene, made machine-readable.

SHOW: worked example (skip if confident)

A research sub-agent finishes reading three papers. Naive return: it pastes a 12K-token summary into the orchestrator’s context. Offline-first return: it writes /tmp/notes/papers_X_Y_Z.md with a 1-page digest and three labelled bullets; the orchestrator sees only the path. The orchestrator can then Read the file selectively, ask a different sub-agent to verify a specific claim, or hand the path to a third agent for cross-citation — and the original 12K of scratch never enters anyone’s context unless explicitly requested.

3.2 Evidence tags: V, B, SCHEMATIC

ORIENT: the big picture before the detail

Tagging every claim is the second offline-first habit. It is the same discipline as labelling a backtest sample IS vs OOS: a guess is not a measurement, and the reader should be able to tell the difference at a glance.

PRIME: new terms, one line each

- **V**: verified — primary source on this machine (settings.json, skill file, framework doc).
- **B**: background — established fact, no fresh citation needed.
- **S**: schematic — illustrative; the shape is real, the numbers are teaching.

The three tags are deliberate, exhaustive, and cheap. Every claim in this book carries one. The only verified number the book leans on is the 1,000,000-token cache window (**V** CLAUDE_CODE_AUTO_COMPACT_WINDOW); all M-targets come from **V** METRICS_FRAMEWORK.md; everything else is either background (**B**) or schematic (**SCHEMATIC**). The tag is a contract with the reader: if it says V, you can re-find the source on this machine; if it says S, the numbers are made up to teach a shape.

SHOW: worked example (skip if confident)

‘Compaction above 0.85 loses the oldest 30% of session context’ is a **SCHEMATIC** claim — the shape is right (older context is summarised, recent context is preserved) but the exact 30% is illustrative. ‘The cache window is 1,000,000 tokens’ is **VERIFIED** — `grep CLAUDE_CODE_AUTO_COMPACT_WINDOW ~/.claude/settings.json` returns the value on this machine.

FADE: same skill, you fill the blanks

Tag these three claims. (a) ‘The cache window is 1,000,000 tokens.’ → _____. (b) ‘Switching models mid-session invalidates the prefix.’ → _____. (c) ‘Peak utilisation rises linearly with 12K tokens per turn.’ → _____.

Answers: (a) V — settings.json on this machine. (b) B — established harness behaviour. (c) S — illustrative teaching number.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Why is a 200K-char tool result in context an anti-pattern?
2. What three things should a handoff doc contain?
3. Which two sources on this machine back the V-tagged claims in this book?

Answers: (1) it stays in the working context permanently and re-pays tokens every turn; it grows the tool-results band faster than anything else. (2) file path, one-line digest, evidence tag. (3) ~/.claude/settings.json (CLAUDE_CODE_AUTO_COMPACT_WINDOW) and o5_QUANTIFICATION/METRICS_FRAMEWORK.md (M1–M12 definitions).

3.3 A reusable handoff-doc template

ORIENT: the big picture before the detail

If you commit to offline-first, you need a stable template for the handoff doc so the sub-agent does not improvise the shape every time. The template is small; the consistency is what makes the orchestrator's life easy.

PRIME: new terms, one line each

- **Header:** 1 line: date, source, evidence tag (V/B/S).
- **Digest:** 1–3 sentences summarising the artefact.
- **Findings:** bulleted list of the load-bearing facts.
- **Open questions:** what you did NOT resolve — for the next agent.

The template fits on screen and prints to one page. Each field earns its place: the **header** is the only line the orchestrator

needs to read; the **digest** is for the next turn's reasoning; the **findings** are the data the next agent verifies against; the **open questions** are the spec for that next agent. Skip a field only if it is genuinely empty — an empty 'open questions' is fine and worth saying so; an empty 'findings' means the sub-agent found nothing and you should probably kill the chain rather than forward the empty result.

Two operational notes. **First**, the file lives in /tmp/notes/, not in the project tree — you do not want the handoff doc committed to the repo. **Second**, name the file with the artefact's stable identifier (paper DOI, dataset name, instrument symbol) so two sub-agents that read the same source write the same path; the orchestrator can dedupe automatically. The Phase-1 audit synthesis **B** called out exactly this naming-discipline gap in the Upwork pipeline audit; the same fix generalises.

RECAP: back to altitude

- Files beat tool results: summarise big outputs into a /tmp/ file and reference by path.
- A handoff doc = path + digest + evidence tag.
- V / B / SCHEMATIC tags make the evidence base legible at a glance.
- The M1 anti-pattern is the dumped file; the fix is offline-first.

SECTION GLOSSARY

Offline-first: produce a file as the artefact; reference it by path; pay for content once. **Handoff doc:** the small file (path + digest + tag) that replaces a tool result in working context. **Evidence tag:** V / B / SCHEMATIC marker attached to a claim to expose its source class.

Part C. Sub-Agent Isolation

Offline-first works because sub-agents exist. This Part is about how to use them properly: the worktree + filter pattern that keeps them honest, JSON-schema StructuredOutput that keeps their returns typed, the memory frontmatter that makes them persistent, and the hot/cold path split that keeps the orchestrator thin. None of this is exotic; all of it is the difference between a clean multi-agent system and a leaky one.

Chapter 4. Worktree + filter + StructuredOutput + hot/cold split

BOTTOM LINE A sub-agent should run in a **worktree**, with a **filter** of allowed tools, return a **typed JSON packet**, and have its **cold-path context never re-imported** by the orchestrator — four independent levers that together keep the orchestrator thin.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- State the four levers and what each one prevents.
- Write a sub-agent frontmatter that exercises all four.
- Decide which sub-agent tasks belong in worktrees vs. in-context.

Prerequisites: Parts A–B.

4.1 The four levers

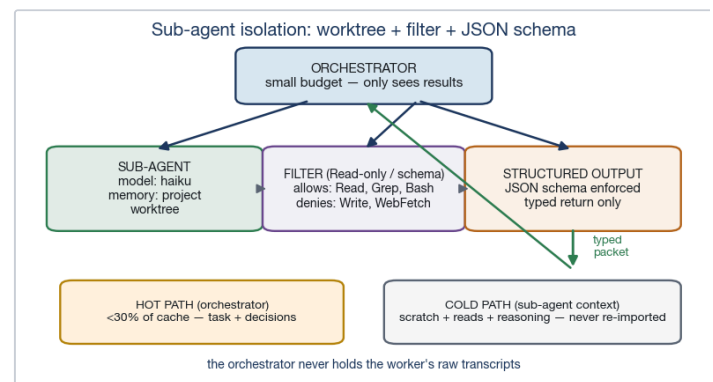
ORIENT: *the big picture before the detail*

Most sub-agent misbehaviour is one of four things: it edited the wrong file, it used a tool it should not have had, it returned a wall of prose the orchestrator now has to parse, or it stayed alive and re-imported its own transcripts. Four independent levers answer each one.

PRIME: *new terms, one line each*

- **Worktree:** an isolated working directory the sub-agent operates in; orchestrator sees the diff, not the path.
- **Filter:** an allow-list of tools the sub-agent can call; deny-by-default for anything that mutates.
- **StructuredOutput:** a JSON schema enforced on the sub-agent's return; typed packet, never prose.
- **Memory frontmatter:** the YAML header of an agent file: model, tools, memory, effort.

orchestrator's working tree. The filter prevents a read-only agent from accidentally calling Write or WebFetch. The StructuredOutput prevents the orchestrator from having to parse free-form prose ('the agent returned 800 tokens of narrative and I need to find the list inside' is a known anti-pattern). The memory frontmatter is the configuration surface for all of the above.



SCHEMATIC Sub-agent isolation: worktree + filter + StructuredOutput. The orchestrator sees only the typed packet; the worker's raw transcripts stay cold.

4.2 The memory frontmatter, end to end

ORIENT: *the big picture before the detail*

The agent file's frontmatter is the contract. Most teams under-use it — they set the name and the prompt and stop. The other five fields are where the discipline lives.

These are independent levers — you can use any one without the others, and a serious production sub-agent uses all four. The worktree prevents destructive edits from landing in the

PRIME: *new terms, one line each*

- **memory: project:** the agent reads/writes a project-scoped notes directory across sessions.
- **model:** which tier the agent runs on (haiku for mechanical, sonnet for assembly, opus for judgment).
- **tools:** the allow-list filter.
- **effort:** low...max deliberation; reserve high for judgment tasks.

a sub-agent that exercises all four levers

```
---
name: data-reviewer
description: Adversary vs. data bugs. Read-only.
model: sonnet          # judgment, but not the heavy
tools: Read, Grep, Glob, Bash # explicit allow-list
memory: project        # persists findings across sessions
effort: high           # this is verification work
---
You are an adversary. Hunt errors,
leakage, and bad assumptions.
Report; do not fix.
```

Every field is doing work. **model:** sonnet routes the agent to the second tier instead of the cheapest or the most expensive — a judgment task that does not need opus. **tools:** is the filter: deny-by-default for Write, WebFetch, and anything else that could mutate state or burn budget. **memory:** project lets the agent keep a notes directory across sessions, so the second time you run it does not start cold. **effort:** high dials deliberation up for verification work where being wrong is expensive. None of this is decoration; every field is a place where the default would have been wrong.

4.3 Hot/cold path split

ORIENT: *the big picture before the detail*

The single most-ignored invariant in multi-agent systems: the orchestrator's working context must stay *hot* — the current task, the latest decisions, the load-bearing rules. The sub-agent's transcripts are *cold* and should never re-enter the orchestrator's window. Returning a wall of prose or pasting a raw tool result violates this; returning a typed packet or a file path preserves it.

The rubric the context-optimizer skill **V** uses is concrete: **hot context < 30% of the cache window**. If the orchestrator is carrying more than that, it is losing the thread — it is starting to behave like one of its own sub-agents, holding the detail instead of the decision. The fix is always the same shape: sub-agent summarises, orchestrator references, file lives in /tmp/.

A simple way to keep this discipline visible: **the orchestrator should not be able to quote any tool result verbatim from the last 5 turns**. If it can, the tool result is in its hot path; if it cannot, the sub-agent did its job. The Phase-1 audit synthesis **B** flagged this exact pattern as the most common silent failure in workspace #6 (Upwork pipeline) and workspace #4 (Medium Christian) — the orchestrator ended up carrying prose it should have delegated, and both peak utilisation and M1 (token efficiency) suffered for it.

SHOW: *worked example (skip if confident)*

You spawn three parallel research sub-agents. Naive: each one returns a 6K-token summary; the orchestrator now holds 18K of new context just from the returns. Hot-path pressure. Offline-first: each one returns a {"path": "/tmp/notes/X.md", "digest": "...", "tag": "V"} packet; the orchestrator holds 300 tokens and a list of paths it can Read on demand. Cold-path transcripts stay in the workers' own windows.

FADE: *same skill, you fill the blanks*

Decide hot or cold. (a) Sub-agent returns the full text of three papers. → _____. (b) Sub-agent returns a JSON packet with three file paths. → _____. (c) Orchestrator pastes the packet into its own context. → _____.

Answers: (a) cold — it bloats hot context; should have written files. (b) hot — small, structured, referenceable. (c) cold — packets belong on disk, not in the orchestrator's window.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Name the four independent levers for sub-agent isolation.
2. What does each field of the memory frontmatter control?
3. What is the hot-context ceiling, and which skill enforces it?

Answers: (1) *worktree, filter, StructuredOutput, hot/cold split*. (2) *model (which tier), tools (filter), memory (persistence), effort (deliberation depth); plus name and description*. (3) *< 30% of cache window, enforced by the context-optimizer skill's scoring rubric*.

COMMON MISTAKE: *you might think X; actually Y*

You might think the four levers are belt-and-braces — you only need one or two. Each one prevents a different failure mode; the failure modes are independent. A read-only filter does not save you from a worker that pastes 8K of transcript back; StructuredOutput does not save you from a worker that edits the wrong file; a worktree does not save you from a worker that returns prose. Stack them.

4.4 A reusable sub-agent frontmatter, annotated

ORIENT: *the big picture before the detail*

A full annotated version of the frontmatter from §4.2 is worth pinning down so each new agent is built from the same template, not improvised.

PRIME: *new terms, one line each*

- **name:** the agent's stable identifier — matches the file name and any /command or routing entry.
- **description:** one sentence, action-oriented, says what the agent *does* and what it *is not*.
- **model:** haiku for mechanical, sonnet for assembly, opus for judgment; reserve the top tier for the tasks that actually need it.
- **tools:** allow-list filter; deny-by-default for any tool that mutates or spends money.
- **memory:** project for state that must persist across sessions; user for state that belongs to one operator; none by default.
- **effort:** low for cheap mechanical turns; max only for the genuinely hard judgment steps.

Three rules of thumb that go with the template. **One:** never set memory without a reason — a persistent memory directory is a maintenance burden and a security surface, so the default is none. **Two:** the description field is the routing hint; if it is vague, the agent gets used for the wrong tasks. Write it the way you would write a commit message: one line, specific, no fluff. **Three:** effort is a hidden cost dial; reserve max for the small number of turns where being wrong is genuinely expensive, and accept that every agent running at max is paying for it.

RECAP: *back to altitude*

- Four levers: worktree, filter, StructuredOutput, hot/cold split — independent but stacked.
- The agent frontmatter is the contract; under-using it is the most common sub-agent mistake.
- Hot context < 30% of cache; sub-agent transcripts are cold and stay cold.
- Return typed packets, not prose; reference files by path, not by content.

SECTION GLOSSARY

Worktree: an isolated working directory for a sub-agent; orchestrator sees the diff. **StructuredOutput:** an enforced JSON schema on the sub-agent's return value. **Hot/cold split:** orchestrator carries decisions and references; sub-agent carries scratch and detail.

Part D. Compaction, Cache & Quantification

The techniques so far keep the window usable; the two chapters in this Part keep it honest. Chapter 5 is the mechanics of compaction — what triggers it, what survives, when to force it. Chapter 6 is the quantification layer — the 12 metrics that tell you whether any of this is actually working. If you only ever read one chapter, make it Chapter 6; without measurement the techniques are aesthetic.

Chapter 5. Compaction triggers, what survives, the 1M economics

BOTTOM LINE Compaction is the harness's summary pass over older turns. With a ✓ 1M cache window, the right discipline is **force /compact at ~0.80 with explicit preserve/drop instructions**, never let it auto-fire at 0.95.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Distinguish auto- vs force-compaction and the cost of each.
- Write preserve/drop instructions that survive a real summary pass.
- Decide what to write to disk before /compact so it survives a worst case.

Prerequisites: Chapters 1–4.

5.1 What compaction is, mechanically

ORIENT: *the big picture before the detail*

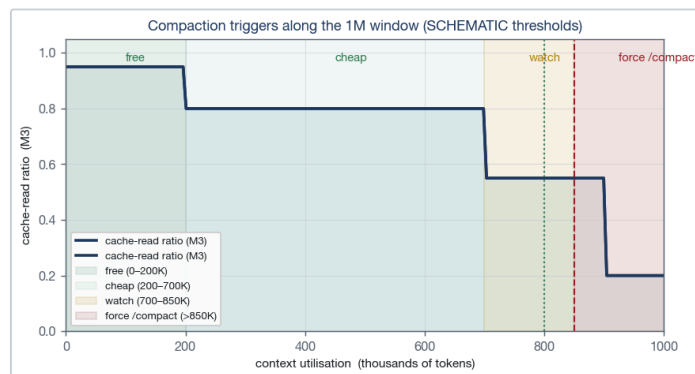
When the harness decides the window is too full to keep taking new turns cleanly, it runs a summarisation pass: older turns get condensed into a single summary block, the raw transcripts are dropped, and the window frees up. The mechanism is mechanical; the choices it makes are not.

PRIME: *new terms, one line each*

- **Auto-compact:** the harness's automatic summary pass; fires around 0.95 on this machine.
- **Force /compact:** an explicit summarisation pass you trigger, usually earlier, with instructions.
- **Preserve list:** the items you mark 'must survive' in the compaction prompt.
- **Drop list:** the items you mark 'can be summarised away' in the compaction prompt.

The reason forcing earlier is cheaper is the shape of the cost curve. A compaction at 0.80 summarises ~150K of older

turns; one at 0.95 summarises ~600K. The output is one summary block either way, but the input is four times larger — and four times more detail is lost. Worse, the harness's default summary does not know which rules and decisions you care about. Your **force /compact** does, because you wrote the preserve list.



SCHEMATIC Compaction triggers along the 1M window. Free (< 200K), cheap (200–700K), watch (700–850K), force (> 850K).

BRIDGE: *from what you already own*

If you have ever tuned a database's checkpoint interval, the shape is familiar: checkpoint too rarely and the recovery cost dominates; checkpoint too often and the checkpoint overhead dominates. The 1M window buys you a wider operating range — the optimal interval is between 0.75 and 0.85 instead of between 0.95 and 0.98 — and a cheaper optimum (your explicit summary, not the harness's). The trade-off is the same; the dial is much friendlier.

5.2 What survives, and what doesn't

ORIENT: *the big picture before the detail*

The hardest thing about compaction is not triggering it; it is knowing what survives a bad one. The default summary keeps recent turns verbatim, summarises older turns, and drops file dumps. That is the wrong default for most workflows.

Two patterns hold up across the research **B**. First, the rules and decisions that matter most are usually the oldest — they were decided early, when the session was still figuring out what it was doing, and they survive into the summary only by luck. Second, file dumps are the easiest thing to drop because they were never summarised in the first place — which is exactly why the offline-first habit (Chapter 3) saves you twice. The fix is to write the preserve list *before* the session gets long, and to keep the load-bearing decisions in a file the orchestrator can re-read at any time.

SHOW: *worked example (skip if confident)*

A two-hour research session has, at peak utilisation 0.83: a 6K-token standing-rules block (decided in turn 4), a 3K-token decision log (last entry turn 38), and a 90K-token PDF dump from turn 22. The harness's default compaction keeps the recent turns, summarises the rules to one sentence, and drops the PDF — you come out the other side with a clean window but the rules are now 'be careful' and the decision log is 'we discussed this'. Force /compact at 0.80 with preserve=[rules full, decision log full], drop=[PDF dump, /tmp/notes/X.md], and the same window-clear costs you two sentences and keeps every load-bearing decision.

5.3 The 1M window economics, again

There is a deeper reason the 1M window matters even though most sessions never approach it: it changes what compaction *means*. On a 200K window, a forced /compact mid-session is expensive enough that teams avoid it and accept the slow degradation. On a 1M window, forced /compact at 0.80 is cheap enough that you can do it as a routine — which is the only reason the 'force /compact at 0.80' rule is a recommendation and not a panic move. The window bought you optionality. Use it.

COMMON MISTAKE: *you might think X; actually Y*

You might think a 1M window means you never need to compact. It means the opposite: you can compact often and selectively, instead of rarely and indiscriminately. The sessions that degrade badly on M3 are not the ones that filled the window — they are the ones that never compacted and let the auto-firing summary lose the load-bearing detail.

5.4 When force /compact loses

ORIENT: *the big picture before the detail*

There is one case where forcing /compact is the wrong move: when the preserve list is longer than the summary can hold. If you have eight standing rules, a thirty-line decision log, a verification table, and a list of open questions, a single summary pass is going to lose something. The fix is not 'do not compact' — it is 'compact in two stages'.

Stage one: /compact with a preserve list limited to the rules and the most recent five decisions. Stage two, in the next session or after a turn or two: pull the verification table and the open questions back from /tmp/notes by path. The two-stage approach keeps the cache-window cost of each compaction pass low, and it never asks one summary to hold what two summaries can carry. The same pattern works for any list longer than the model can be trusted to remember verbatim — which on M3 is anything over a few hundred tokens of detail.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. At what peak utilisation should you force /compact, and why?
2. What two patterns hold up across compaction defaults, per the chapter?
3. Why does a 1M window change the economics of forcing /compact?

Answers: (1) ~0.80–0.85, because the cost curve is steep past that and you lose load-bearing detail in the auto-summary. (2) the oldest turns (rules, decisions) are usually the most important; file dumps are the easiest thing to drop, which is why offline-first pays back twice. (3) forced /compact at 0.80 is cheap enough to do routinely instead of only as a panic move.

5.5 The 1M cache cliff in numbers

ORIENT: the big picture before the detail

A one-line summary of where the cliffs are, in case you skimmed to this chapter first.

The thresholds worth memorising:

Region	Tokens	Behaviour
Free zone	0–200K	cache re-use near-total on Claude; MiniMax-M3: no cache, but per-turn tokens cheap
Cheap zone	200–700K	small premium for new turns on Claude; MiniMax-M3: linear per-turn billing
Watch zone	700–850K	consider force /compact; recall degrading
Force-compact	> 850K	force /compact with preserve list; auto-compact imminent

The exact threshold numbers are SCHEMATIC — they are teaching values derived from the metric framework **v**, not measurements on a particular session. On MiniMax-M3 specifically (probe pass 2026-07-08), the *cache-hit ratio M3 metric is structurally zero* and the per-turn cost is linear in tokens used; the table is presented assuming a cache-aware model where applicable, with the M3 row rewritten for

MiniMax-M3. The *shape* is the load-bearing fact: four regions, increasing per-turn cost, one cliff, one cheap fix before the cliff. Treat the thresholds as defaults; if your session pattern is different (long fan-outs, many small tool results), re-tune them after the first week of measurement, per Chapter 6.

RECAP: back to altitude

- Force /compact at ~0.80 with explicit preserve/drop instructions; never let it auto-fire at 0.95.
- Default compaction summarises rules and drops file dumps — usually the wrong trade.
- Offline-first pays back twice: less to summarise, and the file lives on disk anyway.
- 1M window economics: compaction is cheap enough to do often and selectively.

SECTION GLOSSARY

Auto-compact: the harness’s default summary pass; fires late and summarises aggressively. **Force /compact:** an explicit, earlier compaction pass with your preserve/drop instructions. **Preserve list:** the items that must survive a compaction; write it before the session is long.

Chapter 6. The 12-metric framework

BOTTOM LINE ‘Improve my workflow’ only counts if it is measurable. The framework defines 12 metrics **v** (M1–M12), each with a definition, an instrumentation method, and a good/target threshold. **M1, M2, M3, M6, M7, M10, M12 are auto-instrumented**; the others need a manual log.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- List the 12 metrics and which ones are auto-instrumented.
- Read M1, M2, M3 from the harness and explain what they mean.
- Set up the manual log for M4, M5, M8, M9, M11.
- Avoid the three common measurement pitfalls.

Prerequisites: All previous chapters.

6.1 The 12 metrics, at a glance

ORIENT: the big picture before the detail

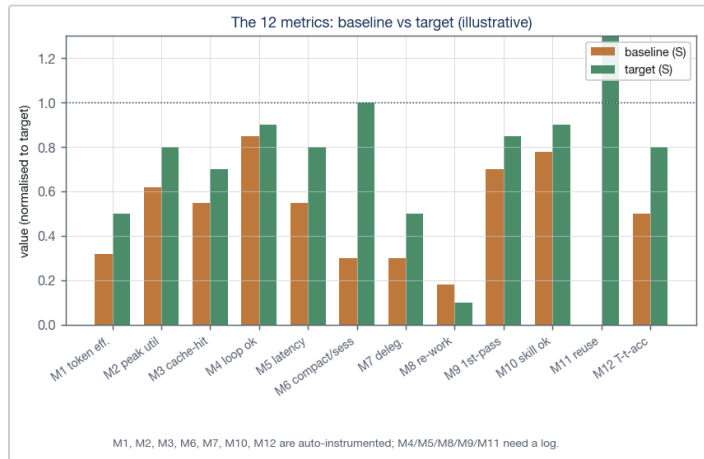
The full table lives in METRICS_FRAMEWORK.md; this chapter is the operator’s cut. Group them by what they measure and you have a usable dashboard.

The 12 metrics, grouped:

Group	Metrics	What they tell you
Token efficiency	M1, M2, M3, M6	are you using the cache well, at what peak, how often you compact
Loop health	M4, M5	are background jobs finishing, at what latency
Orchestration shape	M7, M10	is the orchestrator delegating well, are auto-invoked skills earning their place
Output quality	M8, M9, M11, M12	how often you re-work, how often first-pass lands, time-to-accepted
Headline	M12	wall-clock from first message to acceptance — the bottom line

The headline is M12 (time-to-accepted) and the framework **v** is explicit about what counts as improvement: M12 goes

down **and** at least three other metrics move in the right direction without M8 (re-work rate) going up. The 3-metric floor prevents the ‘faster but sloppier’ win.



SCHEMATIC The 12 metrics: baseline (illustrative) vs target (illustrative). Specific baselines are TBD until the first week of measurement; targets come from METRICS_FRAMEWORK.md.

BRIDGE: *from what you already own*

If you have ever run a backtest with a kill log, you already know the shape: every decision has a row, every row has a verdict, and the log is the only thing standing between the next iteration and the same mistake. The 12-metric framework is the same discipline applied to the workflow itself. The IMPROVEMENT_LOG.md is the kill log; M1–M12 are the verdict columns; the 3-metric floor is the ‘never call it a win unless three numbers moved’ rule.

6.2 The auto-instrumented seven

ORIENT: *the big picture before the detail*

Seven metrics you can read today without any new tooling. M1, M2, M3, M6, M7, M10, and M12 (partially, via session metadata) all surface from harness output you already have.

- PRIME:** *new terms, one line each*
- **M1 token efficiency:** `useful_output_tokens / total_consumed`; good ≥ 0.50 .
 - **M2 peak utilisation:** `max(context_tokens / cache_window)`; good < 0.80 .
 - **M3 cache-hit ratio:** `cache_read / (cache_read + cache_creation + uncached)`; good ≥ 0.70 .
 - **M6 compaction events:** `auto + manual / compact` invocations per session; good ≤ 1 .

Of the seven, three are the ones to glance at every session (M1, M2, M3) and two are the ones to watch over weeks (M6, M7). M10 (skill auto-invoke success rate) and M12 (time-to-

accepted) need a small log on top, which is what the IMPROVEMENT_LOG.md is for. The full instrumentation recipe is in METRICS_FRAMEWORK.md **V** — the columns are definition, instrument, good value, baseline, target.

6.3 The five that need a log

- ORIENT:** *the big picture before the detail*
- Five metrics cannot be read from the harness. They require a manual entry per task, or a small auto-log you set up once. They are also the ones most likely to expose real improvement, because nobody is currently tracking them.
- PRIME:** *new terms, one line each*
- **M4 loop completion:** loops ending in success / loops started; good ≥ 0.90 .
 - **M5 loop latency:** median time from loop start to terminal-state event; loop-type-dependent.
 - **M8 re-work rate:** corrections after first deliverable / total deliverables; good ≤ 0.10 .
 - **M9 first-pass acceptance:** deliverables accepted without revision / total; good ≥ 0.85 .
 - **M11 pattern reuse:** Phase-1 best-practice patterns reused across ≥ 2 workspaces / 30 days; good ≥ 5 .

The log is one line per task: | date | metric | value | target | delta | link |. The framework **V** gives the schema verbatim and the example uses M3 cache-hit ratio(| 2026-07-08 | M3 cache-hit ratio | 0.62 | 0.71 | +0.09 | https://... |). The system-pulse weekly scan picks up new rows automatically. The first week’s job is to log a baseline for M1–M10 before changing anything; the second week’s job is to see what moves.

the M1 worked example, end to end (from METRICS_FRAMEWORK.md)

```
Session: 'build me a backtest memo for strategy X'.
useful  = ~4,200 tokens
total   = 8,900 in + 1,200 out + 600 cache miss = 10,700
M1      = 4,200 / 10,700 = 0.39 (below good)
diagnose: orchestrator re-read 2 long PDFs
fix      : sub-agent pre-summarises the 2 PDFs into a
M1 est   : 0.55 (above good)
```

6.4 The three common pitfalls

ORIENT: *the big picture before the detail*

Measurement has its own failure modes. Three show up repeatedly.

Pitfall 1: measure only the metric you want to improve. If you are optimising M1 (token efficiency) you

can drive it up by producing less output — which is just being less useful. The 3-metric floor in the framework **V** is the fix: M12 must go down and at least three others must move without M8 going up.

Pitfall 2: set baselines after the intervention. The first week of measurement is the baseline; the second week is the first comparison. If you change the system and the log at the same time, you have no control. The framework **V** is explicit: log a baseline *before* changing anything.

Pitfall 3: tag the wrong thing. A 0.85 M9 first-pass acceptance rate looks great until you realise you are counting tasks you accepted by reflex. Pair M9 with M8 (re-work) and M12 (time-to-accepted) — the three together tell you whether ‘accepted’ meant anything.

Each pitfall has the same shape: **a single number, optimistically read.** The framework’s defence against this is the 3-metric floor plus the cross-metric pairing. The Phase-1 audit synthesis **B** called out a related anti-pattern across the workspace portfolio: metrics were claimed (in the AI Engineer Portfolio’s README) but never instrumented. The fix the audit recommended was the scorecard aggregator with a CI gate — exactly the kind of structural defence that the 3-metric floor is in the metrics framework. If a metric is worth tracking, it is worth a check that it is being tracked correctly.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Which seven metrics are auto-instrumented, and which five need a manual log?
2. What is the ‘3-metric floor’ and what is it preventing?
3. Why must you log a baseline *before* the first intervention?

Answers: (1) auto: M1, M2, M3, M6, M7, M10, M12; manual: M4, M5, M8, M9, M11. (2) M12 must drop AND ≥ 3 others move in the right direction without M8 rising — prevents the ‘faster but sloppier’ win. (3) without a baseline you cannot attribute any change to the intervention; you have no control group.

RECAP: back to altitude

- 12 metrics, grouped by token efficiency / loop health / orchestration / output quality.
- M12 (time-to-accepted) is the headline; M8 (re-work) is the brake.
- Seven metrics are auto-instrumented; five need a manual log (M4, M5, M8, M9, M11).
- Three pitfalls: measure only what you want to improve; baseline after the intervention; tag the wrong thing.

SECTION GLOSSARY

3-metric floor: M12 down + ≥ 3 others right + M8 not up — the framework’s definition of improvement.

Baseline: the first week of measurement, logged before any change; the control group. **Auto-instrumented:** metrics that can be read from harness output without a new log.

Part •. Where This Leaves Us

The verdict in one sentence: on a 1M-token cache window, system quality is set by what you let into the window and what survives the next compaction — so spend your attention on context discipline (offline-first files, sub-agent isolation, hot/cold split), and measure it honestly against the 12-metric framework before you change anything.

The evidence base. This manual is grounded in two primary sources on this machine: **V** `~/.claude/settings.json` (`CLAUDE_CODE_AUTO_COMPACT_WINDOW=1000000`) and **V** `05_QUANTIFICATION/METRICS_FRAMEWORK.md` (M1–M12 definitions and targets). The context-optimizer skill **V** supplies the cache-utilisation rubric and the anti-pattern taxonomy; the 8-workspace synthesis **B** (`02_PROJECT_AUDIT/WORKSPACE_MATRIX.md`) supplies the gap list that drove the framework. Every other number in the book is illustrative (**SCHEMATIC**). The deepest lesson is the one the whole book returns to: **a workflow is only as good as its measurement**, and the most senior thing you can do is use the simplest pattern that solves your problem — no more. Discipline context, measure honestly, and escalate only on evidence. That is the whole craft.